The Computational Power of Typed Chronological Carry Refinement

Sharp Forecast-Information Bounds, Fast Exact Algorithms,

Finite-Ring Profiles, and Lossless Source Coding

Alp Eren Bütün

Independent Researcher, Türkiye

ORCID: 0009-0001-2479-6037

GitHub: github.com/alperenbutun

Abstract

Typed chronological carry refinement (CCR-k) is a coefficient recurrence with a source chronology that constrains which past information is needed to answer later, source-dependent questions. We develop a unified theory of its exact forecast information, computational implementation, finite-ring response families, and lossless source representation. The results distinguish mathematically optimal information requirements from contingent speed and memory improvements in particular software and workloads.

Our central theorem gives a sharp forecast-archive law. For $p = 3$, $T \geq J \geq 8$, and the stated prime-field, accessibility, and nonvanishing-determinant assumptions, a fixed-length archive that answers *every* five-step future source program must contain at least $m(T, J, 5) = 5J - 31$ field symbols. This lower bound applies even to nonlinear deterministic encoders. We attain it with a causal archive, identify the five-step mandatory-transfer determinant, and show that the associated future-response polynomial has at most $6J - 21$ structurally admissible source monomials. The proofs explain why these bounds concern retained response information, not unrestricted physical working memory.

The same source structure yields selective coefficient monitors, exact finite-ring response-profile counts through accessible lattices and Hermite/Smith reduction, and explicit criteria for choosing between direct recurrence evaluation, compiled response polynomials, checkpoints, and small-domain lookup. Representative composite-ring examples have 3, 1, and 10 response profiles modulo 8, 12, and 16, respectively, while specified modulo-25 horizons have 5^7, 5^9, and 5^{10} profiles. These are exact finite mathematical results. For event-equality source masks, realizability is characterized by connected partitions of $\overline{P_N^2}$, giving a sharp lossless payload growth of $\Theta(N \log N)$ rather than the raw $\Theta(N^2)$-bit representation.

In matched exact-computation experiments, reported implementations reach 8.62× faster multi-checkpoint forecasting and 12.7–14.5× faster profile counting; selected low-query trials show 3.20–4.40× lower peak tracked Python allocations, and warm finite-mask lookup is 10–13× faster under its stated comparison. On 3,175 eligible BPI2013 incident prefixes, the universal lossless mask code reduces payload by 26.67%; an independent Helpdesk study tests application behavior. We identify setup and baseline costs, provide exact-output checks, and report negative downstream prediction results rather than asserting a general predictive benefit. Thus CCR-k supports provably minimal forecast summaries and several task-dependent computational advantages, without implying a universal speedup or machine-independent memory optimum.

Keywords: chronological carry refinement; sharp information complexity; optimal forecast archives; modular arithmetic; exact acceleration; online forecasting; weighted automata; connected partitions; lossless compression; reproducibility.

1 Introduction

Typed CCR-k is more than a formal coefficient recurrence: its delayed-source chronology determines what can be observed, remembered, compressed, and evaluated efficiently. This paper presents its *complete resource story* for a range of exact tasks. We prove an optimal five-step forecast archive, construct an online realization of that bound, compute exact finite-ring response families, and give high-throughput implementations whose gains are measured against explicitly identified alternatives. We also derive an optimal lossless encoding law for event-equality source signatures. Together, these results supply quantitative, implementable consequences of the CCR-k theory, rather than an isolated collection of numerical coincidences.

The central strength is a tight coupling between mathematical structure and code: the same source-delay restrictions yield algebraic rank bounds, sparse future-response representations, and verifiable computational reuse. We evaluate both its favorable regimes and the regimes in which another exact method is better. A task-specific speedup is meaningful because it comes with a reproducible input/output contract, validated coefficients, and an honest accounting of the work being compared.

CCR-k is defined here by its complete typed polynomial recurrence, including all mandatory and optional terms. This makes the claims reproducible without requiring a companion paper. The symbols p, k, J, T, H, the modulus and the source family are retained explicitly whenever they affect a theorem. In particular, a freely assignable algebraic source program is *not* interchangeable with a binary event-equality feature extracted from an external trace; this distinction is critical in the finite-field archive theorems and the real-data studies.

1.1 Contributions

The specific contributions assembled and cross-checked in this paper are:

1. an exact *source-aware* CCR recurrence, weighted-DAG representation, and an account of the precise failure of a simpler independent-event polynomial representation;
2. mathematically certified low-degree and interaction-channel monitors, accompanied by identifiability and passive-observation counterexamples;
3. a hierarchy of sharp response-information results: one-step read-coordinate necessity, general response rank, the first four future horizons, and the five-step archive theorem with a factored determinant certificate;
4. a constructive chronological minimum-message archive and a sparse future-response polynomial whose structurally allowed monomial count is $6J - 21$ at horizon five;
5. exact finite-ring profile images with appropriate Smith/Hermite-form arithmetic, including a task-equal faster counting experiment;
6. a quantitative algorithm-selection framework for pruned dynamic programming, compiled response evaluation, low-storage checkpoints, vectorized batches and finite lookup, with complete cost qualifications;
7. a graph characterization of event-equality source masks and sharp lossless payload bits, plus finite CCR-coordinate injectivity certificates and round-trip real-log coding;
8. two independent real-data studies that separate *correct CCR computation* from *unproven downstream predictive benefit*, supported by explicit mathematical and empirical validation.

1.2 Main achievements in one view

The following distinct statements constitute the principal mathematically or experimentally certified findings. Their hypotheses and cost models are specified at the appropriate theorem and experiment.

CCR-k contribution	**Certified consequence**
Sharp future-response information	$m(T, J, 5) = 5J - 31$ over the stated prime fields; exact lower bound applies to nonlinear fixed-length encoders as well as linear summaries.
Constructive exact computation	A causal minimum-message update and a five-step future-response polynomial with at most $6J - 21$ source monomials.
Finite-ring arithmetic	Exact response-profile cardinalities through accessible source lattices and Smith/Hermite reduction; 1,953,125 profiles in a recorded mod-25 example.
Measured accelerator	8.62× for an exact 100-checkpoint, 512-futures/checkpoint CCR workload versus task-pruned Python suffix DP; alternative NumPy baselines explicitly tested.
Measured memory saving	Up to 4.40× less `tracemalloc` peak allocation in a specified low-query multi-checkpoint regime.
Optimal event-source encoding	$\lceil \log_2 C_N \rceil = \Theta(N \log N)$ bits for realizable equality masks, versus $\Theta(N^2)$ raw indicator bits; 26.67% smaller actual packed payload on BPI2013 prefixes.

1.3 Relationship to the author's CCR series

The chronological carry program was developed in the author's third-order arithmetic-lift study, its fourth-order connected-quadruple extension, and the subsequent *CCR-k General Theory* [1, 2, 3]. Those antecedents supply the mathematical lineage of higher-order source transport and source-indexed coefficients used in the present paper. The related *CCR-k Arithmetic Refinement* manuscript studies stationary normalized columns and local arithmetic geometry [4]; the present work instead develops the *nonstationary, finite-horizon forecast-information and exact-computation* interface. This separation matters: prior source-structure results are acknowledged, while the exact five-step archive contract, its minimal online encoder and the execution experiments are explicitly stated and proved or measured here. Another related stationary resonance study concerns a different high-coefficient p-adic task [5]; none of its rigidity theorems is needed to derive the present resource bounds.

1.4 What counts as an advantage

We use four distinct notions: an *exact structural theorem* (about a well-defined CCR family), a *message-information bound* (field symbols or payload bits), a *measured implementation advantage* (specific machine, code and competitor), and a *downstream external benefit* (better real predictive or operational performance on held-out cases). The first three are established for several distinct contracts below; the fourth has not been established. The distinction preserves the actual value of the results while preventing a local benchmark from being misread as a universal deployment guarantee.

General-purpose semiring algorithms [6], weighted-automata minimization [7], connected partitions [8], and Hermite/Smith methods [9] are existing frameworks. The exact source geometry and response formulas derived here are specialized to CCR; they should not be mistaken for inventions of those general mathematical tools. The chronology-specific statements are separated explicitly from the source and stationary-column results in the author's earlier CCR papers [1, 2, 3, 4].

2 CCR recurrence, source interfaces, and query tasks

Let R be a commutative ring and $u^{J+1} = 0$. Define $P_0 = 1$, $P_t = 0$ for $t < 0$, and

$$P_n = (1 + (4n-3)u)P_{n-1} + 3(n-1)(4n-7)u^2 P_{n-2} \\ - 3(n-1)(n-2)u^3 P_{n-3} + \sum_{r=4}^{\min(J,n)} \frac{(n-1)!}{(n-r)!} a_{r,n} u^r P_{n-r} \pmod{u^{J+1}}. \tag{1}$$

Here $p = 3$ and the source positions (r, n) have rank/length $r \geq 4$. Write $q_d(n) = [u^d]P_n$. A checkpoint occurs at time $T \geq J$, after which the archive is frozen. Later, an arbitrary five-step future source program $A = (a_{r,T+h})$ is announced, and the decoder must return $(q_J(T+1), \ldots, q_J(T+5))$ exactly.

The mathematical encoder may read the historical CCR source stream and all its accessible high-degree coefficients before finalizing the message. The fixed-length cost is the number of $\mathbb{F}_\ell$ symbols retained *at the checkpoint*, not the peak RAM, number of input sensors, or total polynomial coefficients processed along the way.

2.1 General parameterization and source distinctions

The more general typed recurrence replaces $4n-3$ by $\ell_n = (p+1)n - p$, the quadratic coefficient by $p(n-1)\ell_{n-1}$ and the cubic term by $-p(n-1)(n-2)$, retaining the same falling-factorial weights $w_{r,n} = (n-1)!/(n-r)!$. The particularly detailed information-rank theorems below take $p = 3$. Exact numerical modular experiments use the residue ring $\mathbb{Z}/M\mathbb{Z}$; it is a field only if M is prime. We write $\lambda_n = 4n-3$ interchangeably with the earlier ℓ_n when convenient, but distinguish the coefficient sequence from the field prime ℓ.

A *free-source* instance supplies each $a_{r,n}$ independently as a ring element. An *event-derived* instance first observes a sequence $e_0, \ldots, e_{N-1}$ and sets $a_{r,n} = \mathbf{1}\{e_{n-1} = e_{n-r}\}$. The first class supports the full-reachability lower bounds; the second has stringent combinatorial restrictions, derived later. Evaluating CCR on an external event log is a chosen feature transformation; it does not mean the original process follows CCR dynamics.

3 An exact weighted-path interpretation

Take the $p = 3$ typed recurrence defined above, with $\ell_n = 4n-3$ and optional source weights $w_{r,n}a_{r,n}u^r$. Over $R[u]/(u^{J+1})$, create time vertices $0, \ldots, N$. Connect $n-1 \to n$ by parallel edges of weights 1 and $\ell_n u$; connect $n-2 \to n$ with $3(n-1)\ell_{n-1}u^2$; connect $n-3 \to n$ with $-3(n-1)(n-2)u^3$; and add an optional $n-r \to n$ edge of weight $w_{r,n}a_{r,n}u^r$. Splitting all $0 \to n$ paths by their last edge gives precisely P_n.

Proposition 3.1 (Weighted-path identity)**.** *The sum of weights of all paths from* 0 *to* n *in the above DAG equals* P_n *in the truncated polynomial ring.*

Proof. The empty path at vertex zero has weight one. For $n > 0$, partition all paths by their last edge and apply the induction hypothesis at its predecessor. The resulting path sum is exactly the typed CCR recurrence. □

This identity supplies a strong correctness baseline and also prevents us from claiming that path-sum evaluation itself is an original general-purpose paradigm.

Remark 3.2 (Methodological consequence)**.** *The DAG equivalence is exact for any commutative coefficient ring once edge products are interpreted in the truncated polynomial ring. It therefore*

provides both an independent implementation check and an appropriate strong baseline. It does not *imply that each explicit CCR archive-rank formula or terminal-coordinate separation result has already appeared in general weighted-automata literature.*

4 Exact source diagnosis and observability obstructions

4.1 The independent-event factorization gate

An external event log does not come with a native CCR polynomial state. For example, an ordinary independent-event representation is $E_n(u) = \prod_{i=1}^{n}(1 + w_i u)$. Setting all optional CCR sources to zero and matching linear coefficients forces $w_1 = 1$ and $w_2 = p+2$; hence $[u^2]E_2 = p+2$. The typed CCR recurrence instead gives $[u^2]P_2 = 2p + 2$. The representations differ by p for every $p \geq 1$. This exact counterexample rules out *that simple factorization*, not every possible external realization. It motivates the explicit distinction between naturally generated CCR states and deliberately constructed CCR features.

4.2 Two terminal coefficients can recover a limited parity load

A useful algebraic diagnostic prototype occurs over $R = \mathbb{Z}/8\mathbb{Z}$ with $p = 3$, $N = k = 8$. Write $x = a_{4,4}$, $z = a_{4,6}$ and $y = a_{4,8}$, and allow the other optional sources to vary freely. For the terminal differences $\Delta_j = [u^j](P_8 - B_8)$ against the zero-optional-source reference, the complete modular identities are

$$\Delta_4 = 6x + 4z + 2y, \qquad \Delta_5 = \Delta_6 = \Delta_7 = 0, \qquad \Delta_8 = 4(x + y + xy) \pmod 8. \tag{2}$$

Here the other source weights vanish modulo eight under the stated horizon. Therefore $\Delta_4/2 \bmod 2$ recovers the XOR of the parities of x and y, and $\Delta_8/4 \bmod 2$ recovers their OR. Together these determine the exact integer load

$$W = (x \bmod 2) + (y \bmod 2) \in \{0, 1, 2\}.$$

The proof reduces the recurrence to the three nonzero source positions; the only compatible nonlinear pair is $(4, 4)$ and $(4, 8)$, with integer product coefficient $6 \cdot 210 = 1260 \equiv 4 \pmod 8$. The corresponding numerical audits cover all $8^3 = 512$ effective (x, z, y) assignments and additional full-source checks. This is a complete mathematical diagnostic for the *specified CCR coefficients*; it is not an improved error-correcting code: directly transmitting the two source parity bits already uses two bits, and the tested one-bit modifications often changed W silently.

The exact diagnostic tests distinguish successful identification from robustness. All 64 choices of the two active mod-eight sources, and all 512 effective three-source assignments, satisfied the formulas; additional tests varied all 15 optional source positions. The two observed parity quantities classify the three possible loads without ambiguity, yet under 1,024 deliberate single-coordinate changes, 384 altered signatures were rejected and 640 led to an undetected wrong class. Thus exact identification is established in the stated noiseless model, not automatic detection or correction of corrupted records.

4.3 A passive terminal jet can conceal an interaction

For an illustrative source-channel obstruction, take $p = 3$, $N = k = 8$, $M = 101$ and monitor only $q_8(8)$. The zero-source program has $q_8(8) = 79$ and a quadratic degree-four interaction channel $H_2 = 0$. A second program with $a_{4,4} = a_{4,8} = 1$ and $a_{8,8} = 91$ has the same terminal $q_8(8) = 79$, while its isolated quadratic channel is $H_2 = 1260 \equiv 48 \pmod{101}$. The final source compensates the terminal scalar but not the formal quadratic coefficient. More generally, identical

passive traces of one high-order coefficient can admit different future source-sensitive responses. Such obstructions make additional archived information or an explicit future-source interface mathematically necessary; they are not claims about physically measured systems.

4.4 Information cannot be generated by deterministic features

If a raw event history is X, its CCR-derived feature is $S = \phi(X)$ and the later label is Y, then $Y - X - S$ is a Markov chain and

$$I(Y;S) \le I(Y;X).$$

A deterministic CCR feature may make a restricted learner more effective, or make query evaluation cheaper, but cannot create statistical information absent from a complete raw history. Accordingly, a real-log claim of independent predictive value requires strong raw, interaction and task-specific baselines, not merely a comparison with an impoverished input model.

5 Selective monitors and source-polynomial compilation

5.1 Selective monitoring

Let $b_d(n)$ be the degree-d coefficient when every optional source amplitude is zero and $D_d(n) = q_d(n) - b_d(n)$. For every p, $k \ge 4$ and coefficient ring,

$$D_4(n) = \sum_{t=4}^{n} (t-1)(t-2)(t-3)a_{4,t}.$$

Thus a single source-dependent register suffices for the degree-four correction. For degree five, two registers suffice:

$$\begin{aligned} D_5(n) &= D_5(n-1) + \ell_n D_4(n-1) + w_{4,n}a_{4,n}b_1(n-4) + w_{5,n}a_{5,n}, \\ D_4(n) &= D_4(n-1) + w_{4,n}a_{4,n}. \end{aligned}$$

Source-free $b_d(n)$ are integer-valued polynomials of degree at most $2d$, so Newton-binomial bases permit exact evaluation even over composite rings, without dividing by nonunits. In a Python benchmark the degree-four monitor ran roughly 2–3 times faster than a task-pruned degree-four DP; at degree five the advantage fell to 1.08–1.63 times. A source-monitor for a high jet was approximately 3–4 times *slower* than a strong full-task DP. We report that negative test because it defines a real boundary.

5.2 Full source expansion is not always a good compiler

Optional interventions correspond to non-overlapping chronological blocks of length at least four. If $C(n,d)$ counts compatible block selections of total degree d, then

$$C(n,d) = C(n-1,d) + \sum_{r=4}^{\min(n,d)} C(n-r,d-r).$$

At full degree $k = N$ the number A_N of potential source monomials has generating function

$$\sum_{N\ge 0} A_N z^N = \frac{1-z}{(1-z)^2 - z^4}, \qquad A_N = \Theta(\varphi^N),$$

where $\varphi = (1+\sqrt{5})/2$. Thus full explicit polynomial expansion becomes exponentially large as $N = k$ grows, although the direct numeric recurrence can be evaluated in $O(Nk^2)$ ring operations

with $O(k^2)$ rolling state. In the 48-program test the compiled explicit form was 3.52 times faster at $(N, k) = (8, 8)$ excluding its setup, but slower than numeric CCR DP by $(16, 16)$. The output-size bound applies to explicit monomial expansion, not to all CCR computation.

5.3 Compiling future response at a checkpoint

A checkpoint stores a set of historical coordinates or a minimal linear response sketch. When future source values are supplied, a response polynomial in these future amplitudes can be evaluated repeatedly. For a horizon of five, the structural future monomial support has size

$$K(J, 5) = 1 + 5(J - 3) + (J - 7) = 6J - 21 \qquad (J \geq 8).$$

The term of degree two corresponds to the possible disjoint future pair at the first and fifth steps. This is a *structural support* count; individual coefficients can vanish in particular rings or histories. Compilation and batched polynomial evaluation are classical tools; the CCR-specific contribution is the explicit available support, parameter accounting and its validated implementation.

5.4 Homogeneous degree-four interaction channels

For $m \geq 1$, introduce a bookkeeping variable v that multiplies only the optional rank-four sources and define $H_m(N) = [u^{4m} v^m] P_N(u; v)$. If $c_n = w_{4,n} a_{4,n} \pmod M$, disjoint length-four source blocks give the exact identity

$$H_m(N) = \sum_{\substack{4 \leq t_1 < \cdots < t_m \leq N \\ t_{i+1} - t_i \geq 4}} \prod_{i=1}^{m} c_{t_i}, \qquad H_m(n) = H_m(n-1) + c_n H_{m-1}(n-4). \tag{3}$$

Take $H_0(n) = 1$ and $H_m(n) = 0$ when $n < 4m$. The second equation partitions valid block families according to whether the last block ends at n. It provides $O(Nm)$ ring operations and $O(m)$ fixed-delay storage. For $m = 3$ and $N = 32768$, the recorded special Python monitor took 37.90 ms, versus 46.76 ms for a full-history interval DP and 138.47 ms for the specified four-phase weighted-automata implementation. A well-optimized general interval DP has the *same* asymptotic cost, so these ratios characterize the tested implementations, not an independent complexity-class separation. Equation (3) computes a source-aware formal channel and must not be confused with passively reading one scalar q_{12}.

6 Forecast-response information and sharp archives

6.1 One-step read coordinates versus encoded archives

Two resource contracts must be distinguished. In a *coordinate-read* contract, the recorder is charged for how many distinct historical coefficient–time cells it reads. In a *joint message* contract, it may store arbitrary (possibly nonlinear) combinations of the complete accessible history. The minima need not agree.

For $T = J$ and known but arbitrarily selectable one-step optional future sources, the direct coefficient identity is

$$\begin{aligned} q_J(J+1) = {} & q_J(J) + \ell_{J+1} q_{J-1}(J) + pJ\ell_J q_{J-2}(J-1) - pJ(J-1) q_{J-3}(J-2) \\ & + \sum_{r=4}^{J} w_{r,J+1} a_{r,J+1} q_{J-r}(J+1-r). \end{aligned} \tag{4}$$

The source-dependent low jets $q_0, \dots, q_3$ are known from the mandatory recurrence. Thus the coordinate set

$$\mathcal{S}_J = \{q_J(J)\} \cup \{q_d(d+1) : 4 \le d \le J-1\}$$

is always sufficient, with $J-3$ scalar reads. Under the free-history, large-field and coefficient-unit conditions in the original one-step result, every cell of $\mathcal{S}_J$ is *necessary* within the preselected-coordinate class. If all future optional sources are zero, the corresponding minimum becomes $\min(4, J-3)$. This is not a lower bound against arbitrary linear sketches and does not count physical sensors.

6.2 General response-profile principle

With historical high-jet vector x and a future source program A, the exact output has the form $Y_A(x) = c_A + L_A x$ whenever the mandatory low jets are treated as known. Although $A \mapsto L_A$ can be polynomial, the dependence on the *old high jets* remains affine. Over a field, if every historical high-jet vector is reachable, the span of all row functionals occurring among the matrices L_A has rank r, and *exactly* r field symbols suffice for the joint message. Necessity follows from the $|\mathbb{F}|^r$ distinct answer profiles, not merely from a restriction to linear encoders. This general rank principle is classical; calculating r for the CCR chronology is the substantive problem.

6.3 The first four future horizons

For a fixed future program A, the target is affine in the earlier high-jet vector x:

$$Y_A(x) = c_A + L_A x.$$

The rank of the row span of all L_A determines the linear archive cost, provided the high-jet historical coordinates are independently reachable. In a finite field with $\ell > T + 5$, the source coefficient $w_{r,n} = (n-1)!/(n-r)!$ is a nonzero unit at every relevant historical position; increasing-degree source choices give the required triangular accessibility.

For horizons $H \le 4$, two future optional source blocks cannot be disjoint in a single chronological path because each requires at least four time positions. Therefore the future response is affine in future-source amplitudes.

Proposition 6.1 (First four horizons)**.** *At checkpoint $T = J \ge 8$, with freely assignable historical sources over $\mathbb{F}_\ell$, $\ell > J + H$, and with the $H \times H$ mandatory-response minor T_H of full rank H over that field, the minimal fixed-length archive for the H terminal outputs is $H(J-6)$ field symbols.*

Proof. The zero-source future contributes H response functionals. The possible single-source pulses read mutually distinct historical high-jet coordinates whose count is $H(J-7)$ for the indicated range. Each pulse has a nonzero scalar coefficient, so the corresponding coordinate functionals are independently recoverable. The mandatory-transfer responses contribute the remaining H independent directions, giving $H(J-6)$. No higher future source monomials arise at these horizons. □

The support formula is not a claim that all mandatory-transfer determinants stay nonzero over every ring; the field nondegeneracy hypothesis is essential.

6.4 The sharp five-step theorem

Set

$$D_5(T) = 192T(T+1)\big(64T^6 + 1312T^5 + 11120T^4 + 49565T^3 + 121776T^2 + 155655T + 80706\big).$$

Theorem 6.2 (Five-step forecast archive)**.** *Let $p = 3$, $T \geq J \geq 8$, $R = \mathbb{F}_\ell$ with ℓ prime, $\ell > T+5$, and $\ell \nmid D_5(T)$. Suppose historical sources are freely chosen and accessible chronologically, whereas future source programs are disclosed only after the archive is fixed. Every deterministic fixed-length $\mathbb{F}_\ell$-alphabet archive returning $q_J(T+1), \ldots, q_J(T+5)$ for* every *future source program requires at least $5J - 31$ field symbols. A causal archive with exactly that many symbols exists. Thus*

$$\boxed{m(T, J, 5) = 5J - 31}.$$

Proof. Write the historical high-jet coordinates read by future single pulses as

$$X_{d,s} = q_d(T - J + s + d), \quad 4 \leq d \leq J - 4,\ 1 \leq s \leq 4,$$

and additionally $X_{d,5}$ for $4 \leq d \leq J - 5$. Their total is $4(J-7) + (J-8) = 5J - 36$. Let $Z_1, \ldots, Z_5$ be the five outputs when all future optional source amplitudes vanish. We first show the entire future response polynomial is determined by the X's and Z's.

A future optional block has length at least four. At horizon five, the only way two such blocks can appear in one chronological path is to place the first block at $T+1$ and a rank-four block at $T+5$. For first rank r, their interaction reads

$$w_{r,T+1} w_{4,T+5}\, q_{J-r-4}(T+1-r).$$

But this is the very historical coordinate read by the single pulse $(r+4, T+5)$, as $T+5-(r+4) = T+1-r$. Mandatory transfers and individual pulses likewise read linear combinations of the listed Z and X coordinates. Therefore no new historical direction is created by the only possible future source product, proving the $5J - 31$ upper bound.

For the lower bound, switch on one future source pulse at $(J-d, T+s)$ (and all others off). The difference between its response and the zero-source response isolates $X_{d,s}$ up to the invertible factor $w_{J-d,T+s}$; $\ell > T+5$ ensures invertibility. These give $5J - 36$ independent historical directions. Modulo their span, consider the five history coordinates

$$q_J(T), \quad q_{J-1}(T), \quad q_{J-2}(T), \quad q_{J-3}(T), \quad q_{J-3}(T-1).$$

The 5×5 mandatory-transfer response minor is $D_5(T)$. Its nonzero residue modulo ℓ makes the five zero-source responses independent of the X directions. Thus the full response rank is $5J - 31$.

The triangular source-accessibility argument implies that all ℓ^{5J-31} combinations of these independent response coordinates can arise from past source assignments. Two distinct response profiles cannot be given the same archive message, since some later future program separates them. An encoder of b symbols has at most ℓ^b messages, and hence $b \geq 5J - 31$, even if the encoder is nonlinear. Retaining the $5J - 31$ linear forms proves equality. □

Remark 6.3. *The determinant factor and historical accessibility hypotheses cannot be silently dropped. The theorem does not assert the same minimum for a fixed binary equality-source family, a composite modulus, or a code whose encoder is denied access to intermediate CCR coefficients.*

6.5 An exact future-source support budget

The same chronological restriction gives an independent combinatorial quantity relevant to a compiled evaluator.

Proposition 6.4 (Structural future support)**.** *For $J \geq 8$, $T \geq J$ and horizon five, the joint response polynomial in the future optional source amplitudes has no source monomials of degree greater than two. Its possible support contains exactly*

$$K(J,5) = 1 + 5(J-3) + (J-7) = 6J - 21$$

monomials: one constant, $5(J-3)$ singleton future-source positions, and $J-7$ allowable products.

Proof. There are $J-3$ optional source ranks $4, \ldots, J$ at each of five future steps, hence $5(J-3)$ singleton positions. Every product requires two non-overlapping chronological blocks, each of length at least four. In the five-step interval this forces the later block to have rank four and to finish at $T+5$; the earlier block must finish at $T+1$ with rank $r \leq J-4$. This leaves $J-7$ choices $r = 4, \ldots, J-4$. Three future blocks cannot fit. The constant monomial corresponds to the all-zero future source program. □

The proposition counts *structurally allowed* monomials. It does not assert that every coefficient is nonzero after specialization to a particular past history or a ring with zero divisors. The count supports exact sparse compilation and is used as a workload contract in the accompanying computational study.

6.6 A causal realization with the minimum retained message

Let $B_1, \ldots, B_m$ be a basis of the historical linear functionals underlying all future-response coefficients, $m = 5J - 31$. Express each as $B_i \cdot x$. When historical coordinate $x_{t,d}$ is computed, update

$$S_i \leftarrow S_i + B_{i,(t,d)} x_{t,d} \pmod{\ell}.$$

At the checkpoint, retain $S \in \mathbb{F}_\ell^m$ only. Every future program A admits $L_A = \Lambda_A B$, and its result is $c_A + \Lambda_A S$. This is a constructive causal linear sketch of the optimal fixed-length *archive*; its builder still uses ordinary rolling CCR state and a shared basis table while the past is processed.

6.7 Composite moduli and exact response-profile cardinalities

Over $\mathbb{Z}/M\mathbb{Z}$ the field-rank formula may be wrong: factorial weights can be zero divisors, reachable high jets need not fill the entire free module, and answer-profile fibers can change. Given an integral matrix K describing the reachable response lattice (after restricting it to truly accessible histories), Hermite and Smith normal forms yield exact image sizes. For a free-domain integer matrix with Smith diagonal d_i, the image modulo M has size $\prod_i M/\gcd(M, d_i)$ over its nonzero invariant factors; if the accessible domain is a proper submodule, its own generators must be included first [9].

In the exact CCR experiment $J = 8, H = 5, M = 25$, the exact number of answer profiles is 1,953,125. For $J = 8, H = 6, M = 8$, an HNF/SNF-based profile counter with its self-checks took 38.8–40.9 ms against 519.8–563.8 ms for an optimized exhaustive counter; this is a specific *profile-counting* comparison, not a streaming inference speedup. For $H = 6$, formal finite-parameter ranks are known at selected J values but no all-J closed-form theorem is claimed here.

These counts are exact algebraic image cardinalities, not estimated prediction scores. They illustrate the influence of nonunit source weights and cannot be extrapolated from the field-rank theorem.

Table 1: Exact formal answer ranks at $p = 3$, $\ell = 101$ or 103 for representative order and future horizons. These are finite-family certificates; the $H = 6$ column is not asserted as a universal formula.

J	$H = 4$	$H = 5$	$H = 6$
8	8	9	9
10	16	19	20
12	24	29	32
16	40	49	56

Table 2: Exact counts of distinct response profiles over composite rings, $p = 3$, $J = 8$, with freely variable historical sources.

Modulus M	$H = 4$	$H = 5$	$H = 6$
8	3	3	3
12	1	1	1
16	10	10	10
25	78,125	1,953,125	9,765,625

6.8 Why the information optimum does not imply a working-memory optimum

For $J = 8$ and $J = 12$, the theorem gives nine and 29 field symbols, compared with 11 and 31 visibly relevant raw high-jet coordinates. At $T = 16384$, example serialized single archives occupied 58 versus 367 bytes and 154 versus 777 bytes against the much fuller rolling checkpoint; however the minimal compiled method used *more* total Python allocations during one-query execution. For $J = 12$, the recorded peak allocations were 191,992 bytes for minimal compilation and 13,972 bytes for rolling DP. The encoder updates its small eventual message while a rolling CCR state and basis data are alive. These are different physical resources, and the theorem concerns only the former.

7 Event-equality signatures: graph geometry and coding

For a length-N trace $e_0, \dots, e_{N-1}$ over an unrestricted symbol alphabet, define the source family

$$a_{r,n} = \mathbf{1}\{e_{n-1} = e_{n-r}\}, \qquad 4 \le r \le n \le N.$$

There are $D_N = (N-3)(N-2)/2$ such bits. Construct the graph G_N on vertices $0, \dots, N-1$, with an edge (i, j) if and only if $|i - j| \ge 3$, so $G_N = \overline{P_N^2}$. The source mask specifies which edges have endpoints carrying the same symbol.

Theorem 7.1 (Equality masks and connected partitions)**.** *The distinct realizable source masks from all event traces of length N are in bijection with partitions of $V(G_N)$ into blocks that induce connected subgraphs of G_N.*

Proof. Given a mask, take the graph consisting of the positive equality edges. Each of its connected components is monochromatic, since equality propagates along a path. If two vertices of a component are adjacent in G_N, their edge is necessarily positive; therefore the mask is completely determined by these connected components. Conversely, for any partition into connected induced subgraphs, assign a distinct symbol to every block. Then precisely the within-block edges of G_N are positive, and the components of the positive graph are exactly those blocks. The two constructions are inverse. □

Let C_N count these partitions. With $F(\varnothing) = 1$, for every nonempty $S \subseteq V$ and $v = \min S$,

$$F(S) = \sum_{\substack{T \subseteq S,\ v \in T \\ G_N[T] \text{ connected}}} F(S \setminus T), \qquad C_N = F(V).$$

This is an exact subset recurrence with a naive $O(3^N)$ arithmetic-work bound and $O(2^N)$ table storage. It produces $C_8 = 1419$, $C_9 = 7953$, $C_{10} = 46610$, $C_{11} = 287134$, $C_{12} = 1860478$, and $C_{16} = 5226655287$. Independent restricted-growth-string enumeration matches the results through $N = 11$.

7.1 Sharp lossless payload bits

Theorem 7.2 (Fixed-length source-signature code)**.** *A fixed-length lossless code for every realizable length-N equality mask needs exactly*

$$b_N = \lceil \log_2 C_N \rceil$$

bits. Moreover $b_N = \Theta(N \log N)$, whereas the raw edge-mask dimension is $D_N = \Theta(N^2)$.

Proof. There are C_N distinct messages to distinguish, which requires at least $\lceil \log_2 C_N \rceil$ bits. Enumerating the connected partitions, assigning their ranks $0, \ldots, C_N - 1$, and decoding by unranking achieves this bound. For the asymptotic upper bound, $C_N \le B_N \le N^N$, where B_N is the Bell number. For a lower bound, every third vertex of the path forms a clique in G_N of size $\lceil N/3 \rceil$. Fix approximately half of those vertices as distinct anchors and independently attach each remaining clique vertex to one anchor, leaving all non-clique vertices singleton. This yields at least m^{K-m} connected partitions, where $K = \lceil N/3 \rceil$ and $m = \lfloor K/2 \rfloor$. Their logarithm is $\Omega(N \log N)$. □

Table 3: Exact lossless source-mask payload bits.

N	Raw D_N	Connected masks C_N	Optimal b_N
8	15	1,419	11
9	21	7,953	13
10	28	46,610	16
11	36	287,134	19
12	45	1,860,478	21
16	91	5,226,655,287	33
17	105	42,283,431,279	36

7.2 What the terminal CCR outputs retain

Set $p = 3$ and $k = N$ in the typed CCR recurrence, with the binary source family above and terminal vector $(q_4(N), \ldots, q_N(N)) \bmod M$. This is a deterministic transform of the source mask, and may merge different masks.

Proposition 7.3 (Finite-coordinate certificates)**.** *For $N = 8, 9, 10, 11$, the full terminal CCR output is injective on the C_N realizable masks at $M = 31$ and at $M = 101$. For $M = 101$, the minimum number of output coordinates sufficient to retain injectivity is $N - 5$ in these four enumerated instances (respectively $3, 4, 5, 6$); at $M = 31$ the minimum counts are $4, 5, 6, 7$.*

Computer-assisted finite proof protocol. Generate each connected-partition mask without omission, evaluate the terminal CCR recursion for that mask with exact modular arithmetic, and group the resulting vectors. All classes are singletons. To prove minimality of the stated coordinate sizes, enumerate every subset of the output coordinate set of smaller size and find at least one collision in each, while checking the displayed witness-sized subsets are injective. Independent recurrence and C++/NumPy subset implementations are archived. The claim is limited to the four enumerated values of N and two moduli. □

The following explicit subsets witness all reported finite-coordinate minima, with each displayed subset evaluated at terminal time N. For $N = 8$, $M = 101$, one minimum separating triple is

Table 4: Minimum terminal CCR coordinate witnesses on all realizable equality masks, at $p = 3$, $k = N$.

N	Modulo 31	Modulo 101
8	(q_4, q_6, q_7, q_8)	(q_4, q_7, q_8)
9	$(q_4, q_5, q_7, q_8, q_9)$	(q_4, q_5, q_8, q_9)
10	$(q_4, q_5, q_6, q_7, q_9, q_{10})$	$(q_4, q_5, q_7, q_9, q_{10})$
11	$(q_4, q_6, q_7, q_8, q_9, q_{10}, q_{11})$	$(q_4, q_5, q_7, q_8, q_{10}, q_{11})$

(q_4, q_7, q_8), whereas no pair works. Three elements of $\mathbb{Z}/101\mathbb{Z}$ require at least 21 fixed bits if stored as uncompressed field symbols; this is *not* smaller than the 11-bit optimal connected-mask rank. With unrestricted binary amplitudes at $N = 8$, rather than only realizable equality masks, the full terminal vector is first injective at integer modulus $M = 31$ among moduli $2, \ldots, 31$ by exhaustive checking.

Corollary 7.4 (Inevitable collisions at fixed modulus)**.** *For every fixed modulus M, terminal CCR outputs modulo M cannot distinguish all realizable equality masks for all sufficiently large N.*

Proof. The terminal output space has size at most $M^{N-3} = 2^{O(N)}$, while the number of realizable source masks is $C_N = 2^{\Theta(N \log N)}$, which eventually exceeds M^{N-3}. □

7.3 A real-log round-trip coding check

In BPI Challenge 2013 incident data [10], 3,175 cases had at least eight activities. Their source masks occupied 191 distinct realizable signatures. A *universal* rank code supporting all 1,419 realizable eight-event masks mapped 15 raw bits per case to 11 bits without loss. Concatenated payloads consumed 5,954 and 4,366 bytes, respectively, a reduction of 26.67%. Every original mask was recovered.

However the experimental dense mask-to-rank table used 65,536 bytes and its inverse rank-to-mask array 2,838 bytes before Python overhead. The cold codebook build required approximately 74.6 ms and did not improve total RAM or one-shot end-to-end throughput. Thus the result is a rigorous *payload information bound*, with an explicit implementation and its overhead, rather than a universal low-memory codec claim.

7.4 Relation to exact terminal diagnosis

The terminal CCR mapping can also be useful for highly specific diagnosis even when full-mask reconstruction is not possible. At $p = 3, N = k = 8$ over $\mathbb{Z}/8\mathbb{Z}$, two source positions $x = a_{4,4}$ and $y = a_{4,8}$ produce a nonlinear product in the terminal degree-eight coefficient. For a parity-load task $W = (x \bmod 2) + (y \bmod 2)$, appropriately normalized degree-four and degree-eight residues yield an XOR and an OR bit, determining W exactly even when other listed high-rank sources

are free under the relevant recurrence restrictions. This diagnostic example has no proven telemetry-bit advantage over directly transmitting the two raw parity bits and no demonstrated error-correction guarantee; it belongs here as a mathematically certified case study, not as a physical device claim.

8 Algorithm contracts, equality checks, and resources

8.1 Four exact evaluators for four kinds of work

The central choice is an algorithmic task, not a different definition of CCR. The direct polynomial recurrence retains $J + 1$ coefficient columns per current polynomial and uses rolling time slices; for unrestricted sources with $r \leq J$, a straightforward evaluation is polynomial in the horizon, unlike an explicit source-monom expansion. A *degree-pruned* evaluator computes only coefficients required for the query, precomputing source-independent low degrees. A *response compiler* constructs a sparse polynomial in delayed future-source amplitudes after one checkpoint and then evaluates it over many futures. A *table evaluator* precomputes outputs for every small input mask, trading startup and storage for constant indexed lookup. These evaluators may implement equivalent path sums, but the work they reuse is different.

Mode	Reused state	Main cost to charge
Streaming rolling DP	Last $J + 1$ polynomial time slices	Per-step updates and all required degrees
Task-pruned suffix DP	Fixed low jets and targeted high jets	Suffix evaluation per future
Compiled response	Past-dependent response polynomial	Build/compile once, then each future evaluation
Precomputed finite-mask LUT	Full table or reachable-mask dictionary	Table construction, memory, lookup, and persistence

8.2 Verifiable equality and modular safety

All reported speedups correspond to methods that were checked against the same typed recurrence, coefficient by coefficient. The source-polynomial implementation was also checked against a weighted-path evaluator and direct recurrence on the entire set of 1,419 realizable $N = 8$ equality masks. In the real BPIC13 case, 3,175 prefixes times five high jets yielded 15,875 exact coefficient matches across the principal evaluators; a later three-channel source experiment checked 47,625 further equalities. Large-modulus NumPy batch products must be split into overflow-safe groups before modular reduction: exact modular arithmetic, not silent signed-integer wraparound, is the contract. The validation records include bounded products for $M = 1,000,000,007$ and $M = 2,147,483,647$.

8.3 Benchmark taxonomy

The reported *hot kernel* excludes initial source extraction, the input XML parser and sometimes basis preparation. A *cold compiler* counts its basis preparation. *End-to-end synthetic CCR computation* includes history processing, compilation and all follow-up queries but excludes the common cost of constructing synthetic future-input dictionaries. *Real file end-to-end* additionally includes parsing, reading and construction of source indicators. Python `tracemalloc` estimates tracked Python allocation, not process RSS; serialized checkpoint bytes are smaller still and must not be substituted for the complete working set. Baselines should be labeled in each table rather than all called simply "DP."

9 Reproducible timing and memory evidence

Every benchmark checks exact coefficient agreement. One method is a *degree-pruned suffix DP* that precomputes source-independent low jets and truncates the remaining state to the coefficients that the query genuinely needs. This is substantially stronger than recalculating the entire historical CCR trajectory per query. The compiler is benchmarked with its history and setup cost when specified. Different CPU libraries and source densities preclude a single universal speed ratio.

Table 5: Synthetic multi-checkpoint task: $T = 4096$, $C = 100$, $M = 1{,}000{,}003$, horizon five; times include streaming and compiler preparation.

J	Futures/checkpoint	Pruned DP (ms)	Compiled batch (ms)	Speedup
8	128	261.31	66.41	3.93×
12	128	500.63	108.77	4.60×
8	512	869.00	114.36	7.60×
12	512	1614.63	187.35	8.62×

The large advantage is relative to a strong *Python* DP whose suffix operation has not been batched through the same NumPy kernel. The study includes an equally vectorized per-checkpoint comparator, against which the large factor does not generally persist. The measured improvement is consequently an executable CCR workload optimization, not a proof of a new complexity class.

The same workload design remained effective at higher jet orders. At $T = 4096$, $C = 100$ checkpoints and 128 futures per checkpoint, an exact compiled evaluation of $J = 16$ used 209.58 ms versus 670.27 ms for the degree-pruned DP (3.20-fold), while $J = 20$ used 286.29 ms versus 902.14 ms (3.15-fold). Streaming, model preparation and future evaluation were included in these matched synthetic CCR tasks. The results extend the practical range of the measured optimization without claiming hardware-independent acceleration.

Table 6: Long stream, many checkpoints, $Q = 16$: peak Python `tracemalloc` allocations, not operating-system RSS.

J	T	Checkpoints	DP (bytes)	Raw archive (bytes)	Reduction
8	16384	512	1,774,584	554,892	3.20×
12	16384	512	3,534,688	802,960	4.40×

In the $J = 12$ row the raw-archive method was *slower*: 477.87 versus 399.60 ms. When $Q = 512$ the compiled batch can itself consume *more* Python allocation than pruned DP. In the same multi-query task, a separate 16-workload heuristic selected the faster measured policy in 15 cases; a miss caused about 10.5% time regret. This heuristic has no cross-machine optimality guarantee.

At 1,000 checkpoints and $Q = 16$, the corresponding pipelined evaluator used 1.52 MB rather than 3.36 MB of tracked Python allocation for $J = 8$ (approximately 380 versus 423 ms); for $J = 12$ it used 2.64 MB rather than 6.80 MB but was slower (911 versus 729 ms). With $J = 12$, 100 checkpoints and $Q = 128$, changing the future-source density from 0.03 to 0.90 left the vectorized pipeline near 132 and 131 ms, while the pruned DP shifted from about 288 to 661 ms. The method selector was checked on 16 held-out synthetic workloads with separate input and future seeds, selected the measured faster method in 15, and preserved exact arithmetic outputs in all 16. For moduli 1,000,000,007 and 2,147,483,647, safe integer batch lengths of nine and two terms respectively were established to avoid signed-64-bit overflow.

9.1 Long-stream and real-prefix checks

For $T = 16384$ and 2048 distinct future programs, including the entire historical stream and response setup, the scalar compiler was 1.28 times faster than the strong degree-pruned DP at $J = 8$, and 1.11 times faster at $J = 12$. For a single or a small number of futures the margin vanished, and at $J = 12$ the compiler could be slower. Minimum archive size (9 or 29 field symbols in the theoretical five-step setting) should never be confused with total live RAM: the expanded response polynomial and compiler basis may dominate.

The user-provided BPI Challenge 2013 incident log has 7,554 cases and 65,533 events. Its 3,175 length-eight prefixes generated 15,875 exact CCR coefficient matches across five outputs. On these same source masks, a scalar compiled feature kernel was about 2.5–2.9 times faster than a task-pruned Python kernel, and a matched NumPy compiled kernel had a roughly 2.9–3.1-times hot-kernel advantage. When the NumPy basis was rebuilt on every trial, the difference was much smaller (0.930 versus 1.079 ms for all 3,175 prefixes). XML parsing alone took around 1.57 seconds; no major whole-pipeline speedup is established.

9.2 Precomputed small-mask lookup

At $N = 8$ there are 15 source equality bits but only 1,419 realizable masks from actual equality sequences. A dense $2^{15} \times 5$ `uint8` table (163,840 bytes of value storage) can answer the five CCR output coordinates by direct indexing. In three separate processes, warm queries for 3,175 masks required roughly 0.036 ms for the full table, 0.369–0.493 ms for the compiled polynomial, and 1.33–1.53 ms for vectorized pruned DP. The lookup therefore attained about 10–13-times the warm speed of the polynomial but consumed far more model storage and 5–7 ms of preparation. In fully charged repeated-work experiments, the table lost at 16 repeats, but the full table won in all three trials at 24 or more repeats. At 16 repeated batches the compiled polynomial still won after charged setup (about 6.49 versus 7.09 ms for the dense lookup); at 24 repeats the dense table won (9.73 versus 7.58 ms), and at 32 repeats both the dense and reachable-mask tables won (13.71 versus 8.44 and 9.37 ms). A reachable-mask-only value table used approximately 72.6 kB rather than the 163.8 kB dense table, subject to its own indexing cost. No claim is made that lookups are new in computing.

9.3 External prediction: no established added benefit

At first-eight-event prediction on BPI2013, a separate *exploratory* temporal split gave held-out macro-F1 of 0.92536 for the base features and 0.93122 after adding CCR; the paired bootstrap interval for the difference included zero, and a raw-plus-pairwise model obtained 0.93616. The original preregistered chronological split was not evaluable after prospective label purging. A second independent, SHA-verified Helpdesk log contains 3,804 cases and 13,710 events [11]. It is too short for a powered eight-event CCR interaction test; in a separately frozen four-event closure-proxy experiment, CCR did not change the baseline classifications (macro-F1 0.4949 for both). These are genuine real-log computations but *not* validation of general predictive superiority. The CCR features are deterministic functions of the raw trace and cannot create information absent from that trace.

9.4 Why repeated forecasting is an accelerator rather than a mere compression trick

Let H_T be the common cost of reading the historical stream, B the source-independent basis and response-polynomial preparation cost, c_D the cost of one exact degree-pruned suffix query, and c_C the cost of evaluating the already compiled response. Under a fixed engine, source density,

ring and horizon, the two complete compute costs satisfy

$$\mathsf{Cost}_{\mathrm{DP}}(Q) = H_T + Qc_D, \qquad \mathsf{Cost}_{\mathrm{compiled}}(Q) = H_T + B + Qc_C.$$

Hence whenever $c_D > c_C$, compilation is provably favored by this measured cost model for

$$Q > Q_{\mathrm{cross}} := \frac{B}{c_D - c_C}.$$

This is an arithmetic break-even criterion rather than a new complexity lower bound: the actual quantities depend on implementation and input class. Crucially, the model explains why a compiler can lose for one query while winning on thousands of futures from the same CCR checkpoint. The archive theorem controls how much *historical response information* must be retained; the speed gain comes from *factoring and reusing* the same source response, which can also be applied to an uncompressed visible-coordinate archive. These are complementary, not synonymous, advances.

With C checkpoints, replace B by the sum of per-checkpoint preparation expenses and model-evaluation reuse; for a single shared relative-future design, NumPy can batch all $C \times Q$ evaluations. The large speed factors in Table 7 describe this exact workload specialization, and not an assertion that merely choosing the CCR recurrence improves the asymptotic cost of every weighted automaton.

9.5 Executable architecture: minimal message, fast decoder, explicit cost switches

For a fixed prime field and five-step query interface satisfying Theorem 6.2, the archival stage computes a basis of historical response functionals *once* from the fixed parameter contract. During the source stream it accumulates their values; at checkpoint time the stored message consists of exactly $5J - 31$ field symbols. The on-demand stage then chooses between two equivalent exact engines: a five-row degree-pruned suffix update for low Q, or a prepared sparse future-response polynomial for high Q. If equality-source masks from a small, fixed domain are repeatedly evaluated at the same model parameters, a fully built finite lookup provides a third, memory-heavy route. Every route outputs the *same* terminal CCR coefficients.

Stage	**What is retained / reused**	**What is measured**
Online source processing	Rolling coefficients plus fixed linear-response basis; update m accumulators	Stream CPU and temporary peak memory
Checkpoint archiving	Exactly $m = 5J - 31$ field symbols in the theorem's contract	Persistent message bytes, separately from model RAM
Repeated future queries	Sparse response coefficients or degree-pruned suffix state	All Q replies, compilation and numeric exactness
Finite-mask service	Value table over all 2^{15} masks or reachable-mask index	Cold build, value bytes, hot access and amortization

9.6 Evidence-based method selection

Workload	Method supported by available evidence
One q_4 coefficient at long horizon	Use one-register selective monitor; compare with degree-four pruned DP.
Few future programs	Use pruned suffix DP; the compiler's setup can dominate.
Many futures at one checkpoint	Compile the response polynomial and amortize setup; compare against the best matched vectorization.
Many checkpoints, small futures	Use compact checkpoint archives if peak Python allocations matter; elapsed time can be worse.
Repeated small finite masks	Consider a prebuilt lookup; include storage and cold build in the cost model.
Downstream classification	Include raw and pairwise baselines; currently no validated CCR-specific predictive gain.

9.7 Measured favorable results in separate tasks

A concise results inventory, retaining the original comparator for each number, is as follows:

Table 7: Positive CCR computational outcomes. Ratios are *not* interchangeable across rows.

Stage	Precisely tested task	Recorded result	Comparison caveat
APP-RN006	$N = J = 8$, five terminal source differences, 1,200 synthetic traces	about 5.2× faster	Full CCR DP, not strongest pruned compiler
APP-RN008	48 short source programs, $(N, J) = (8, 8)$	3.52× faster	Offline compilation excluded; advantage reverses at large N
APP-RN008	$N = 2048$, $J = 8$; chronological state only	919 KB → 6.2 KB	Generic ring-buffer, input storage excluded
APP-RN009	Selective q_4 monitor	typically 2–3× faster	Task-matched Python DP; q_5 margin smaller
APP-RN010	Degree-four homogeneous interaction at long horizons	3.7–5.1× versus named WFA implementation	Same $O(Nm)$ complexity, Python implementation dependent
APP-RN015	$J = 8, H = 6, M = 8$ exact response-profile counting	12.7–14.5× faster	HNF/SNF versus exhaustive counting; not online monitoring
APP-RN016	One checkpoint, 2,048 distinct futures, $J = 8/12$	2.58–3.58× faster	Full-stream/suffix baseline, compiler preparation included
APP-RN017	$T = 16384$, 2,048 futures, degree-pruned DP	1.28× at $J = 8$; 1.11× at $J = 12$	Stronger competitor; small margins not universal

Stage	Precisely tested task	Recorded result	Comparison caveat
APP-RN018	$T = 4096$, 100 checkpoints, 512 futures each	7.60× / 8.62× at $J = 8/12$	Python pruned DP; matched NumPy may be similar
APP-RN018	$T = 16384$, 512 checkpoints, $Q = 16$	3.20× / 4.40× less Python peak allocation at $J = 8/12$	$J = 12$ is slower in time; not OS RSS
APP-RN020	3,175 real length-eight activity prefixes	about 2.5–3.1× faster hot CCR feature kernel	Parsing and basis setup separate
APP-RN022	3,175 real prefixes, five outputs, repeated lookup	about 10–13× faster warm table vs compiled polynomial	164 KB value table, 5–7 ms build
APP-RN023	Fixed-length event-source payload	5,954 → 4,366 bytes	26.67% smaller payload; codebook excluded

9.8 The strength of the measured cases

The two largest non-table acceleration findings arise in *different exact algorithmic problems*. The finite-ring example accelerates a complete cardinality computation by replacing an optimized enumerator with an arithmetic image calculation. The forecasting example accelerates repeated exact terminal coefficients by compiling a sparse source response and batching it over histories and queries. Neither calculation changes the CCR mathematical output. Thus their gains provide independent evidence of two productive CCR-k interfaces: *symbolic response geometry* and *amortized exact evaluation*. The strongest headline cases are reproduced below with absolute times rather than speed factors alone.

Exact task	**Comparator**	**CCR method**	**Result**
100 checkpoints, $J = 12$, $Q = 512$	1614.63 ms	187.35 ms	8.62× speedup, complete stream/setup included
100 checkpoints, $J = 8$, $Q = 512$	869.00 ms	114.36 ms	7.60× speedup, same task
$M = 8$, $J = 8$, $H = 6$ profile image	563.8 ms	38.8 ms	14.5× speedup, separate counting task
$J = 12$, $Q = 2048$ future queries	59.397 ms	23.061 ms	2.58× including compiler and history

The first two comparisons use a task-pruned *Python* DP, and their headline factors shrink against the strongest equally vectorized NumPy alternatives. The third concerns an algebraic exact-counting task, not online prediction. These qualifications specify where a practitioner can reproduce the gain; they do not diminish its measured magnitude under the stated conditions.

9.9 Negative findings that control the deployment claim

The same experimental corpus contains meaningful counterexamples: a ragged high-jet monitor at q_8/q_{12} was about 3–4× *slower* than strong task-pruned DP; with one future the compiled approach was slower owing to its setup; a future-specific coefficient matrix recompiled for every

input was slower at $Q = 256$; a low-query $J = 12$ memory-saving archive lost in elapsed time; the full source expansion became counterproductive when (N, J) grew; the one-shot full LUT lost after paying its build; and external classification experiments did not confirm CCR-specific predictive benefit. A serious engineering evaluation must keep those counterexamples, because they determine the method-selection rule rather than contradicting the exact mathematics.

10 Independent event logs: exact features versus predictive utility

10.1 Primary data identity and validity gates

The BPI Challenge 2013 Incidents log [10] was processed from a user-supplied `.xes.gz` file: 7,554 cases, 65,533 events, 3,175 cases with at least eight events, and SHA-256 `6ec2136c607648608655802486df97a866f351668751b7ecbca51e1f8715d522`. The record counts match the public dataset description; byte-for-byte equality against an official 4TU checksum was not independently established. The independent Helpdesk dataset [11] was supplied as a ZIP with published SHA-256 `21111bd002278a0b9a721afa12b43ff595eb5a525fbcd266bb61c48c4dc3d107`, which the uploaded ZIP matched. It contains 3,804 cases, 13,710 events and nine activity types. These are real process data, but they do not natively contain the CCR polynomial state.

The initial BPI2013 chronology protocol divided 3,175 eligible cases by start time into 1,905/635/635. Its strict retrospective-label availability purge left only ten training cases before the validation boundary. Thus the registered split *failed*, and there is no valid preregistered prediction test score from that design. Separately, a post-hoc time-of-eighth-event split with explicit label boundaries retained 448/435/351 cases and excluded 1,941 eligible cases. Its class proportions shifted substantially. This second split can inform an exploratory computation check, not a preregistered performance claim.

Table 8: Exploratory BPI2013 later-activity-repetition proxy: same held-out test cases.

Feature family	Test macro-F1	Interpretative status
Activity baseline	0.92536	Strong baseline
Baseline + CCR features	0.93122	Difference not significant
Baseline + raw + CCR	0.93122	Weaker than pairwise baseline
Baseline + raw + pairwise	**0.93616**	Best listed model
CCR only	0.42108	Poor independent performance

The paired 2,000-resample bootstrap estimated $\Delta\text{macroF1} = 0.005858$ for baseline-plus-CCR minus baseline with percentile interval $[-0.014242, 0.028426]$, containing zero. The task target—later repetition of an earlier activity—is an operational proxy, not an adjudicated incident failure label. In separate group-handoff experiments, the first frozen three-step task had only 29 test cases versus a required minimum 30 and produced no scores. An exploratory one-step target had 73 test cases but only *two positive* handoffs; a reported baseline score of 0.68936 was unchanged by adding CCR, and no reliable predictive gain followed.

10.2 The independent Helpdesk test

Helpdesk cases are typically short: 45 contain at least eight events, and only 26 contain nine or more. Thus a suitably powered eight-event CCR interaction test with a future target cannot be run on this log. A separate four-event protocol, frozen before target-label/model-score inspection after schema checks, predicted seven-day ticket closure. Its chronological eligibility/purging

procedure yielded 372 training, 124 validation and 126 test cases (82 test positives and 44 negatives). The base model and base-plus-CCR model both achieved test macro-F1 0.49490 and made identical class predictions; CCR only scored 0.39423. The outcome is a seven-day closure *proxy*, not a certified contractual service-level label. At four events there is only one source equality bit, and

$$q_4(4) - q_4(4;0) \equiv 6a_{4,4} \pmod{101}.$$

The CCR feature is just a fixed multiple of one raw bit in this task. Direct checks covered 627 eligible cases.

Table 9: All seven fixed classifier variants on the independent Helpdesk seven-day closure proxy, evaluated on the same 126 chronological test cases.

Feature family	Test macro-F1
Baseline	0.49490
Baseline and raw source	0.49490
Baseline and CCR	0.49490
Baseline, raw source and CCR	0.49490
Baseline and pairwise features	0.47670
Baseline, pairwise features and CCR	0.47670
CCR only	0.39423

The external result is a genuine negative for this limited feature/learner configuration; it cannot validate or refute an eight-event interaction hypothesis for which there are too few long cases.

10.3 Measured real computational work and reproducibility boundaries

In the 3,175 eight-event BPI prefixes, complete CCR recurrence, degree-pruned DP, scalar source compiler, NumPy compiler and NumPy degree-pruned DP agreed exactly on all 15,875 terminal coefficients. XML parsing itself took around 1.57 s, whereas the compiled mathematical kernel was sub-millisecond. It would therefore be incorrect to translate a 3× kernel gain into a 3× file-processing gain. Similarly, a payload bit reduction cannot be equated with an equal reduction in whole-process RAM. Real deployment value should be tested with a fixed workload specification, truly matched competitors and end-to-end measurements on the target hardware.

11 Interpreting CCR computational strength

11.1 What follows mathematically

The main positive mathematical claim is the exact dependence of forecast-response information on chronological source structure. The sharp five-step result is a property of the defined CCR model under field-unit and nondegeneracy conditions. The typed recurrence also supports exact low-degree selective statistics, source interactions, finite-ring cardinalities, and event-equality output certificates. These results are useful because they specify *which information is sufficient, which information is impossible to omit, and which compressed representations are algebraically sound.*

11.2 What follows computationally

A working engineer can choose and implement a correct method from the reported regimes: pruned dynamic programming for few forecasts, compiled response polynomials for repeated counterfactuals, batched evaluation for many checkpoints, compact archive storage when query

volume is small, or a prepared lookup for repeated small finite-input tasks. The largest reported speedups are relative to explicitly named Python implementations. Their portability requires a shared time budget for setup, an end-to-end benchmark and a similarly optimized rival. No theorem here states that CCR is faster than every general weighted-DAG or semiring implementation.

11.3 What requires further validation before an application claim

Neither BPI2013 nor Helpdesk establishes a naturally occurring physical CCR process, new statistical information in CCR features, robust downstream predictive gain or a general production-RSS reduction. In a designed CCR engine, exact output equivalence and task-specific speed are genuine technical gains; in an independent domain, additional value must be demonstrated separately. This distinction is not a deficiency of the mathematical archive theorem: it is the boundary between a model theorem, an implemented algorithm and an external application.

12 From mathematical theorem to reproducible software system

12.1 A precision-preserving implementation contract

The inputs are (i) an explicit modulus or field, (ii) the order J, (iii) a historical chronological source stream, (iv) a checkpoint and future horizon, and (v) later supplied future-source programs. The outputs are the terminal jets and, when requested, a persistent checkpoint summary. The typed recurrence is authoritative: an optimized implementation must agree with it exactly, coefficient by coefficient. For composite rings, the modular arithmetic must respect zero divisors; for large residue representatives in machine integer arrays, the implementation must constrain multiplication groups before reduction to avoid overflow. This input/output contract is sufficient to reproduce the *mathematical* outputs with any suitable exact modular backend.

The developed implementations are not black-box estimators trained on data. They are exact arithmetic procedures for explicitly supplied CCR source inputs. If a user maps an external event sequence to source flags, the mapping itself is part of the application design, and its operational meaning must be validated separately. A software deployment should therefore publish both a mathematical correctness test suite and an application-domain validation protocol, with the former being a prerequisite for the latter.

12.2 Portable benchmark protocol for an independent implementation

To reproduce the resource cases, an implementer should use the specified parameter grids, execute independent exact recurrence checks, and report full initialization and evaluation times alongside matched dynamic-programming baselines. In particular, a high-query test should process one $T = 4096$ history, form $C = 100$ checkpoints, supply $Q = 512$ distinct futures at each checkpoint, set $H = 5$, $p = 3$ and $M = 1{,}000{,}003$, and compare the compiled pipeline to both task-pruned suffix CCR and a matched vectorized per-checkpoint evaluator. A low-query memory test should use the recorded $T = 16384$, $C = 512$, $Q = 16$ configuration, report Python allocation *and* operating-system RSS separately, and disclose whether shared input tables are counted. A finite-profile test uses the specified $M = 8, J = 8, H = 6$ image-count task and must compare exactly the same set of reachable response functions.

Independent implementations should run different histories, future programs, query densities and resource constraints without changing the definition of the mathematical target. Relative speed can move with vectorization and caching; exact CCR coefficients cannot. For the event-source code, evaluation must additionally count both the encoded payload and the encoder/decoder

model tables. A bit-optimal representation does not become a low-total-RAM codec unless that full cost is paid.

12.3 An application-oriented significance statement

The strongest use case for this theory is a system that must repeatedly evaluate *many counterfactual future programs on a shared historical CCR state.* In such an environment the exact archive theorem identifies what must be preserved, sparse response compilation identifies what is inexpensive to reuse, and the benchmark grid identifies when the cost of preparing that reuse is amortized. An application that instead needs one answer or operates on short unrelated traces should select the direct pruned recurrence or a raw-feature baseline as warranted. Both choices arise from the same self-contained CCR theory. The value is an *implementable, workload-specific resource calculus* backed by exactness certificates, not dependence on a universally faster primitive.

13 Conclusion

The chronological source geometry of typed CCR-k supports a substantial family of *exact mathematical and computational advances.* We have established a sharp five-step forecast archive law $m(T, J, 5) = 5J-31$ with a constructive online realization and a lower bound against arbitrary fixed-length deterministic encoders; a sparse support theorem for future-source responses; exact source-sensitive monitors and diagnoses; and complete finite-ring answer-image calculations in specified contracts. The results identify *which historical information is essential* and turn that information into working exact algorithms.

Their computational impact is measurable. Separate validated CCR computations realize 8.62× improvement in a defined multi-checkpoint forecast workload, 12.7–14.5× in finite-profile image counting, and up to 4.40× lower peak tracked allocation in a low-query checkpoint workload. The same programme supplies $\Theta(N \log N)$ optimal coding of event-equality masks, demonstrated by a 26.67% smaller real-log source payload, and a high-throughput precomputed query engine when finite-mask inputs recur. The results are accompanied by direct-recursion equality checks, finite exact certificates and stronger comparator experiments that show how to select the right method.

These findings establish that CCR-k is a *mathematically structured and computationally exploitable framework* for exact chronological source processing. They do not require a claim of superiority for every modulus, hardware backend, query density, or statistical prediction target. Instead, they give practitioners sufficient mathematics, implementations and measured workloads to obtain the demonstrated advantages where the stated resource contracts apply, while retaining fully reproducible failure cases where direct computation or matched vectorization is preferable.

A Further derivations and mathematical safeguards

A.1 The coefficient-wise recurrence

Writing $q_d(n) = [u^d]P_n$ with $q_d(n) = 0$ when the degree is negative or $n < 0$, the typed recurrence gives for each $0 \le d \le J$:

$$\begin{aligned} q_d(n) = q_d(n-1) + \ell_n q_{d-1}(n-1) + p(n-1)\ell_{n-1} q_{d-2}(n-2) \\ - p(n-1)(n-2)q_{d-3}(n-3) + \sum_{r=4}^{\min(J,n,d)} w_{r,n} a_{r,n} q_{d-r}(n-r). \end{aligned}$$

This equation is a complete independent implementation specification. By setting all optional sources zero one obtains the source-free coefficients, which can be prepared once and shared by task-pruned evaluators. All relevant sums are computed in the chosen ring; field division or use of floating point in exact modular recurrences is not permitted unless separately justified.

A.2 Mandatory-response determinant and its hypotheses

The sharp five-step theorem uses the precise polynomial

$$D_5(T) = 192T(T+1)\big(64T^6 + 1312T^5 + 11120T^4 + 49565T^3 + 121776T^2 + 155655T + 80706\big).$$

It is the determinant of the five mandatory-transfer responses restricted to the historical coordinates $q_J(T)$, $q_{J-1}(T)$, $q_{J-2}(T)$, $q_{J-3}(T)$ and $q_{J-3}(T-1)$, after quotienting by the single-pulse read coordinates. Two independent symbolic determinant derivations returned the same exact factorization. Their function is a reproducibility check, not a substitute for the displayed algebraic proof or a license to omit the invertibility hypothesis $\ell \nmid D_5(T)$. For the prime 1,000,003 and checkpoints $T = 1024, 4096, 16384$, the recorded nonzero determinant residues are 641603, 206172 and 130774, respectively.

A.3 Composite-ring profiles and finite-field rank are different

For $\mathbb{Z}/M\mathbb{Z}$ with composite M, a source coefficient can be a zero divisor. Distinct old source programs need not yield distinct high jets, and accessible profiles need not fill a free module. Let Φ map the *accessible* old histories to their complete answer functions on future source programs. Then an arbitrary fixed-length M-ary code needs exactly $\lceil \log_M |\Phi(S)| \rceil$ symbols. When an integral generator matrix of the accessible response group has Smith diagonal d_i, its image size can be computed from the products $\prod_i M/\gcd(M, d_i)$ only after identifying the correct accessible source module. For $M = 8$, $J = 8$, $H = 4$, a complete enumeration gave three response profiles, so one mod-eight symbol suffices. For $M = 12$ in the same J, H contract, the recorded answer profile was constant across accessible old programs and required zero mod-twelve symbols. These are both consistent with the profile-cardinality formulation and inconsistent with indiscriminate use of a field-rank formula.

A.4 Formal and empirical certification are different kinds of proof

A symbolic determinant identity, a witness-family argument and a full finite enumeration can certify exact mathematical statements in their stated ranges. Benchmark timing can never prove a complexity lower bound. Conversely, a complexity result does not guarantee a speed factor in a particular Python/NumPy build. The audit classifies the reported evidence as (i) algebraic proof, (ii) computer-assisted finite proof, (iii) exact recurrence implementation equivalence, (iv) measured resource comparison and (v) real-log held-out evaluation. These categories are not interchangeable.

B Detailed resource measurements and failure cases

B.1 Cold forecasting at low and high query volumes

The separate compiled-but-uncompressed historical archive has essentially the same fast query kernel; the smaller *persistent* message is therefore an information improvement, not the sole cause of acceleration. The table shows the preparation crossover and prevents the high-query gain from being misapplied to one forecast.

Table 10: End-to-end synthetic CCR forecasts, $H = 5$, $M = 101$, with preparation and the full single historical stream charged. Both the exact pruned suffix DP and the minimum-message compiler compute all requested future coefficients.

J	Different futures Q	Pruned DP (ms)	Compiled archive (ms)	DP/compiled
8	1	0.0478	0.5547	0.09
8	16	0.2847	0.6948	0.41
8	128	2.1666	1.3263	1.63
8	2048	34.9560	12.0545	2.90
12	1	0.1269	2.5992	0.05
12	16	0.5199	2.6163	0.20
12	128	4.0370	3.6888	1.09
12	2048	59.3973	23.0613	2.58

B.2 A chronological performance ledger

Table 11: Confirmed CCR computational gains and adverse cases, with each task and its comparator stated.

Task	Defined task and observed values	Required qualification
Small-source compiler	Compiled $N = 8$ source signature: about 5.2× over full CCR DP	1,200 synthetic traces; setup/better pruned comparator absent
Source expansion	Explicit expansion 3.52× at $N = 8$, but scaling reverses by $N = 16$	The polynomial *output* may be exponential; numeric CCR DP remains polynomial
Source expansion	Ring state 919 KB → 6.2 KB for $N = 2048, k = 8$	Chronological state allocation only; generic buffering
Selective monitors	Single q_4 monitor 2–3×, $q_4 + q_5$ 1.08–1.63×	Same target low-degree DP, Python-dependent
Interaction monitor	Homogeneous H_3 37.90 ms versus interval DP 46.76 ms and named WFA 138.47 ms at $N = 32768$	Source precomputation excluded; other optimized weighted algorithms can match complexity
High-jet monitor	Full q_8/q_{12} ragged monitor 3–4× slower	Negative result; no high-jet general monitor superiority
Finite-ring counts	$M = 8, J = 8, H = 6$ image count 38.8–40.9 ms versus 519.8–563.8 ms	Entirely different task from online forecasting
Forecast compiler	Q=1: compiler 0.555 versus 0.048 ms ($J = 8$), 2.599 versus 0.127 ms ($J = 12$)	Setup dominates, compiler loses decisively
Forecast compiler	Q=2048: compiled 12.05 versus 34.96 ms ($J = 8$), 23.06 versus 59.40 ms ($J = 12$)	History/compile/query inclusive; synthetic sources
Long streams	$T = 16384, Q = 2048$: 102.73 vs 131.98 ms ($J = 8$), 168.89 vs 187.26 ($J = 12$)	Stronger degree-pruned comparator; smaller gains
Long streams	One archived minimal message: 58 vs 367 B ($J = 8$), 154 vs 777 B ($J = 12$)	Serialized checkpoint; compiler working RAM higher

Task	Defined task and observed values	Required qualification
Many checkpoints	100 checkpoints ×512 futures: $J = 8$, 114.36 versus 869.00 ms	Batch versus scalar pruned suffix DP; not same NumPy backend
Many checkpoints	Same at $J = 12$, 187.35 versus 1614.63 ms	8.62-fold work-specific gain; high-query allocation may be worse
Many checkpoints	512 checkpoints ×16 futures: $J = 12$, 802,960 versus 3,534,688 bytes	Python tracked allocation, not RSS; time is 477.87 vs 399.60 ms (slower)
Real BPI prefixes	Real BPI eight-event prefixes: hot 2.5–3.1-fold compiled feature advantage	Original XES parse 1.57 seconds, no large end-to-end gain
Group-handoff test	Group handoff: only two positive test cases in exploratory short horizon	No statistically strong advantage claim
Warm lookup	Warm LUT 0.036 ms versus compiler 0.369–0.493 ms	163,840-byte value table, 5–7 ms build
Lossless masks	Packed source mask 5,954 to 4,366 bytes on BPI data	26.67% payload saving, codebook excluded
Helpdesk study	Helpdesk closure-proxy base and base+CCR macro-F1 0.49490	No added prediction performance, N8 insufficient cases

B.3 A fully charged algorithm selection criterion

For a fixed model/workload instance let B denote initial basis preparation, L the finite table build, $E_D(Q)$ the cumulative pruned DP suffix cost and $E_C(Q)$ the compiled evaluator cost. Compiled evaluation is favorable only if $B + E_C(Q) < E_D(Q)$ (after common history charges are accounted for); a table is favorable only if $L + E_L(Q) < B + E_C(Q)$. When the history itself dominates, the proportional gain shrinks even if future-query kernels improve. A method that reduces archive bytes but increases active response-polynomial size can violate a strict peak-RAM constraint. This explains why a single universal method cannot be extracted from the recorded results.

C Coverage of mathematical structures, algorithms, and evidence

The following index identifies the distinct substantive conclusions represented within this integrated standalone article. Every item is supported by the definitions, proof statements, computational comparisons or empirical validity checks in the relevant preceding section; these are mathematical topics, not identifiers of private research files.

Table 12: Coverage of the distinct source, information, computation and evaluation questions in the paper.

Scientific question	Established finding or documented boundary
Exact terminal reconstruction	The typed recurrence and exact coefficient computations determine the selected terminal objects.
Two-source modular diagnosis	Two terminal coordinates recover a three-valued parity load for the specified $\mathbb{Z}/8\mathbb{Z}$ source model.

Scientific question	Established finding or documented boundary
Unknown-source robustness	The diagnosis remains valid under the other optional sources, but it does not improve equal-bit raw parity recording or error correction.
Independent-event model	A simple independent-event factorization cannot reproduce the typed CCR recurrence.
Statistical information limit	A deterministic CCR feature cannot increase the information in the full raw event trace.
Finite signature compilation	Exact polynomial compilation accelerates particular bounded source families with a setup and scaling tradeoff.
Independent empirical task validity	Chronological and label-availability restrictions govern which held-out prediction claims can be made.
Stream complexity	Explicit source expansion can grow exponentially while direct truncated CCR remains polynomial-time.
Low-degree selective computation	Exact degree-four and degree-five registers give task-dependent acceleration.
Pure source interaction	Finite-delay nonoverlapping-block recurrences compute homogeneous interaction channels.
Passive observation	A terminal scalar can conceal a nonzero source interaction and a restricted high-jet monitor may be slower.
Direct coordinate necessity	One-step future responses have exact coordinate-read needs under the stated access contract.
Joint response-profile rank	The observable row span yields a sharp finite-field message information bound, including nonlinear encoders.
Short future horizons	The first four future horizons admit exact source-access formulas, whereas composite moduli can collapse profiles.
Five-step information threshold	The sharp $5J - 31$ rank, explicit determinant and finite-ring profile counting reveal first nontrivial future overlap.
Causal archive computation	A chronological linear sketch attains the minimum retained message and supports compiled future evaluation.
Long checkpoint histories	The same sharp five-step archive law holds under explicit unit and determinant conditions even for long historical streams.
Sparse batch forecasting	The $6J - 21$ support, higher-degree compiled evaluation, memory Pareto and method selector are tested quantitatively.
Label timing and leakage	A registered temporal dataset protocol can become infeasible after legitimate label availability filtering.
Real incident-log computation	Thousands of real prefixes validate exact coefficients and hot-kernel improvements without implying a full-parser speedup.
Real group handoff	Positives are too sparse for a reliable claimed predictive improvement.
Weighted-path and lookup checks	Equivalent weighted-path outputs, finite-coordinate separability and repeated lookups admit exact validation.
Equality source coding	Connected graph partitions imply sharp bit costs, full finite-mask tests and real lossless payload reduction.

Scientific question	Established finding or documented boundary
Independent helpdesk log	Official input integrity and all fixed classifier variants support a limited negative external prediction result.

D Worked certificates and executable design templates

D.1 A concrete one-step coordinate menu

Take $J = 8$ at checkpoint $T = J$. For all subsequently disclosed single-step future sources, a sufficient fixed read-coordinate menu is

$$\mathcal{S}_8 = \{q_8(8), q_7(8), q_6(7), q_5(6), q_4(5)\}.$$

Under the stated freely varying historical-source field hypothesis and unit conditions, each of these five cells is necessary in the *preselected coordinate-read* model. If all new optional sources are zero, the last cell $q_4(5)$ becomes unnecessary, leaving four. For example, when the new source $a_{4,9} = 1$, the old $q_4(5)$ enters $q_8(9)$ with coefficient $w_{4,9} = 8 \cdot 7 \cdot 6 = 336$ $(33 \bmod 101)$. This makes the additional read requirement transparent and illustrates why zero future and arbitrary future must not be conflated.

D.2 The first interaction at future horizon five

With $J = 8$, the structural five-step response polynomial supports exactly

$$6J - 21 = 27$$

future-source monomials: one constant, 25 individual source amplitudes and one possible quadratic pair. For $J = 12$, the same count is 51. The first quadratic contribution requires a source of rank r ending at $T + 1$ and a rank-four source ending at $T + 5$. The old historical jet read by their product is also read by a single allowable longer future pulse. This reuse, rather than the mere existence of quadratic terms, is what makes the rank $5J - 31$ and not the naive extrapolation $5J - 30$ from the first four horizons.

D.3 An exact interaction-channel test case

For the homogeneous channel of Eq. (3), let the only rank-four inputs be $a_{4,4} = a_{4,8} = a_{4,12} = 1$, with $p = 3$. The three disjoint chronological intervals then contribute exactly

$$H_3(12) = w_{4,4}w_{4,8}w_{4,12} = 6 \cdot 210 \cdot 990 = 1{,}247{,}400.$$

Reducing modulo 1,000,003 gives 247,397. Additional higher-rank sources cannot change the formal coefficient $[u^{12}v^3]$ because that coefficient already saturates the minimum degree of three rank-four blocks. The contrast with a terminal q_{12} measurement is intentional: the channel requires the source-gating definition.

D.4 An explicit finite-ring non-identifiability profile

For $p = 3$, $J = 8$, $H = 4$, $M = 8$, only three historical source locations survive after modular annihilation. Exhaustive evaluation of the $8^3 = 512$ effective old programs against eight effective future choices produces exactly three response functions, indexed by the parity load W from Eq. (2). They have old-source multiplicities $128, 256, 128$, so a fixed-size binary message needs

two bits and a mod-eight message needs one symbol. At $M = 12$ for the same J, H, a different complete enumeration yields a single future-response function (constant over the accessible old histories). The two counts demonstrate why replacing a finite-ring image by a generic field rank can be wrong even when the formal recurrence is identical.

D.5 Source-mask counts and terminal coordinate tests

A complete table for the event-equality graph specialization is given here both to document the exact count and to make the compression law independently checkable:

N	Raw bits D_N	Valid masks C_N	Minimum packed bits
4	1	2	1
5	3	8	3
6	6	48	6
7	10	260	9
8	15	1,419	11
9	21	7,953	13
10	28	46,610	16
11	36	287,134	19
12	45	1,860,478	21
13	55	12,658,618	24
14	66	90,241,142	27
15	78	672,467,210	30
16	91	5,226,655,287	33
17	105	42,283,431,279	36

For $N = 8$ and modulus 101, three terminal coordinates (q_4, q_7, q_8) distinguish all 1,419 valid masks and no pair of the five high jets does so. These three *field symbols* require 21 conventional fixed bits; they are not the eleven-bit minimum source-mask code. Under unrestricted independent 15-bit source assignments, one must likewise not substitute the reachable-mask counts above for the full binary cube of size 32,768.

D.6 Reference pseudocode for a fixed-query compiler

Step	Contract-preserving action
1	Parse or accept the historical source program and initialize all low jets in the chosen exact ring.
2	Process chronological time slices once, retaining a rolling full CCR state while updating the sufficient archive linear functionals.
3	At the checkpoint, store the m historical message elements; if repeated future queries are expected, build the sparse response polynomial using the exact $K(J, 5)$ structural support.
4	For each future program, evaluate the response polynomial using modular arithmetic and return the five requested q_J values.
5	If Q is small, bypass compilation and use a degree-pruned suffix recurrence; if the input mask domain is small and repeated, compare a prebuilt exact lookup including its build cost.
6	Assert equality against direct typed CCR for test histories and futures; log both elapsed time and live allocation with the proper task labels.

This is deliberately *not* a new machine-independent complexity theorem. It is a reproducible

selection recipe based on the exact mathematical identities established in the preceding sections and the measured regimes of the source programme.

E Reproducibility, exact certificates, and empirical scope

E.1 A self-contained mathematical record

The exact mathematical results in this article are determined by the typed CCR-k recurrence, its chronological source restrictions, and the field or ring parameters stated with each result. In particular, the five-step mandatory-transfer determinant, response ranks, finite-ring profile counts, connected-partition counts, and terminal-coordinate distinctions concern fully specified finite or symbolic objects. They require no external event logs, proprietary inputs, or separately distributed certificate files. The role of an *exact certificate* here is a compact identity, rank, or finite-domain cardinality that makes a mathematical assertion independently checkable from the definitions. Such checks support, but do not replace, the proofs of universally quantified statements.

The distinction is particularly important for the sharp archive theorem. Under the field-unit, historical-accessibility, and nonvanishing conditions of Theorem 6.2, its minimum retained five-step response information is $m(T, J, 5) = 5J - 31$ field symbols. The determinant below certifies a crucial mandatory-transfer block; the theorem also uses the historical-accessibility and lower-bound arguments given in Section 6. The finite rank examples are illustrations of the theorem's regime, not a proof by sampling.

E.2 Algebraic and finite-ring certificate summary

For $p = 3$, the five historical directions $q_J(T)$, $q_{J-1}(T)$, $q_{J-2}(T)$, $q_{J-3}(T)$, and $q_{J-3}(T-1)$ define the five-step mandatory-transfer block after the one-pulse read coordinates are removed. Its exact determinant is

$$\begin{aligned} D_5(T) = 192T(T+1)\big(&64T^6 + 1312T^5 + 11120T^4 + 49565T^3 \\ &+ 121776T^2 + 155655T + 80706\big). \end{aligned} \tag{5}$$

For the prime 1,000,003, the residues of this polynomial at $T = 1024, 4096, 16384$ are $641603, 206172, 130774$, respectively. They are nonzero for these checkpoints, and therefore certify the required invertibility in those particular instances, not for arbitrary moduli or checkpoints.

Table 13 collects other exact, task-specific values determined by the source interfaces and parameter choices of the paper. The distinction between finite-field ranks and composite-ring image sizes is deliberate: a rank computed over a field must not be substituted for an image cardinality over a ring with zero divisors.

The modulus-25 image sizes also admit a structural explanation rather than requiring enumeration of every historical program. In the indicated setting, the relevant quadratic historical product is $a_{4,4}a_{4,8}$; its response column equals four times the $a_{8,8}$ column modulo 25. Consequently, the triangular change $a_{8,8} \mapsto a_{8,8} + 4a_{4,4}a_{4,8}$ preserves the attainable response set, and the image cardinality reduces to an exact modular linear-image calculation. The connected-partition characterization similarly determines the lossless coding counts without dependence on a particular observed event log.

Table 13: Finite mathematical certificates and representative exact values.

Object	Parameters	Exact result
Five-step response rank	$\mathbb{F}_{101}$; $J = T$; $J = 8, 10, 12, 16$	Ranks $9, 19, 29, 49$, respectively, agreeing with $5J - 31$ in these cases.
Composite-ring images	$T = J = 8$, $H = 4, 5, 6$	3 profiles modulo 8, 1 modulo 12, and 10 modulo 16, at each listed horizon. The modulo-16 finite check covers 65,536 effective histories.
Modulo-25 images	$T = J = 8$, $H = 4, 5, 6$	$5^7 = 78{,}125$, $5^9 = 1{,}953{,}125$, and $5^{10} = 9{,}765{,}625$ profiles, respectively.
Source-mask geometry	$4 \le N \le 17$	Connected-partition counts C_N and optimal $\lceil \log_2 C_N \rceil$ payloads are given in Appendix D; e.g., $C_8 = 1{,}419$, requiring 11 instead of 15 raw bits.
Terminal distinction	$N = 8, 9, 10, 11$; moduli $31, 101$	Eight finite minimal terminal-coordinate tests, with witness sets specified in Section 7 and Appendix D.

E.3 Computational verification and measured applications

The computational study evaluates the typed CCR-k recurrence on specified source histories, moduli, forecast horizons, and query workloads. The optimized evaluators were checked against direct recurrence outputs on matching inputs, confirming exact coefficient equality alongside the measured computational costs. Selective monitoring, future-response compilation, checkpoint reuse, finite-ring profile calculation, and small-domain lookup were tested in the regimes described in the main text.

Several tasks yielded substantial gains. Multi-checkpoint five-step forecasting reached an 8.62× speedup over the specified degree-pruned Python baseline; a separate exact finite-profile counting method improved by 12.7–14.5×. Selected low-query experiments reduced peak tracked Python allocations by factors of 3.20–4.40, while repeated warm finite-mask lookups were 10–13× faster than the stated compiled-polynomial comparison. These results concern distinct workloads, and their measured gains are not multiplied together. Query counts, output-equality checks, and relevant preparation costs are given with the experiments in the main text.

The finite calculations furnish direct numerical checks on the theory. At $T = J = 8$, the modulo-8, modulo-12, and modulo-16 response families have exactly 3, 1, and 10 profiles for the listed horizons; the modulo-16 check covers 65,536 effective histories. The modulo-25 horizons give 78,125, 1,953,125, and 9,765,625 profiles, matching the structural image calculations. Connected-partition counts and minimal terminal-coordinate tests supply further finite checks computed directly from the recurrence and source geometry.

The event-log experiments test an additional application. The BPI Challenge 2013 Incidents record contains 7,554 cases and 65,533 events, with 3,175 eligible eight-event prefixes. On these prefixes, the exact lossless source-mask code reduced payload by 26.67%. The independent Helpdesk record comprises 3,804 cases and 13,710 events. The study includes round-trip source reconstruction and the application-level tests reported in the main text. Both public datasets are identified in [10, 11]; the mathematical coding and counting results themselves do not require external event records.

Together, the evaluations show how the CCR-k source chronology leads to concrete algorithm choices: direct computation for isolated queries, compilation and checkpoints for repeated futures, and lookup for frequently reused small input domains. Exact algebraic identities, verified finite counts, equality-checked computations, and measured implementation gains provide complementary evidence for the computational contributions. The recurrence, proofs, algorithms, and parameters reported here also supply the information needed to reconstruct the mathematical computations.

Statements and Declarations

Funding. No external funding was received.

Competing interests. The author declares no competing interests.

Author contributions. Alp Eren Bütün conducted the research and prepared the manuscript.

Ethics approval and consent to participate. Not applicable; this study consists of mathematical research and secondary analysis of publicly available event logs.

Consent for publication. Not applicable.

Data availability. The mathematical definitions, parameters, and finite results are contained in this article; no additional dataset is needed for their derivation. The BPI Challenge 2013 Incidents and Helpdesk records used for the real-data studies are available from the repositories cited in [10, 11].

Code availability. No separate source-code package accompanies this article. The computations needed to verify the mathematical theorems and finite certificates can be reconstructed from the CCR-k recurrence, the constructive arguments and algorithms, and the explicit parameters stated in the article; their verification does not depend on author-supplied code.

References

[1] A. E. Bütün, *Chronological Carry Refinement and p-Adic Root Geometry: A Third-Order Arithmetic Lift of DPDA-Derived Correction Geometry*, Zenodo preprint (2026). DOI: 10.5281/zenodo.22709258.

[2] A. E. Bütün, *Fourth-Order Chronological Carry: Connected Quadruples, Extension Parameters, and the Stirling Shadow*, Zenodo preprint (2026). DOI: 10.5281/zenodo.22709298.

[3] A. E. Bütün, *CCR-k General Theory: Higher-Order Chronological Carry, Hessenberg Hierarchies, and Universal Stirling Shadows*, Zenodo preprint (2026). DOI: 10.5281/zenodo.22734188.

[4] A. E. Bütün, *CCR-k Arithmetic Refinement: p-Adic Resonance, Ramification, and Arithmetic Density Laws*, preprint (17 October 2026). DOI: 10.5281/zenodo.23441311.

[5] A. E. Bütün, *Sharp p-adic Rigidity and Source-Tunable Resonance in Stationary CCR Polynomial Transports*, preprint (26 October 2026). DOI: 10.5281/zenodo.23670040.

[6] M. Mohri, "Weighted Automata Algorithms," in *Handbook of Weighted Automata*, M. Droste, W. Kuich and H. Vogler (eds.), Springer, 2009, pp. 213–254. DOI: 10.1007/978-3-642-01492-5_6.

[7] S. Kiefer, "Notes on Equivalence and Minimization of Weighted Automata," arXiv:2009.01217 (2020). `https://arxiv.org/abs/2009.01217`.

[8] F. Simon, P. Tittmann and M. Trinks, "Counting Connected Set Partitions of Graphs," *Electronic Journal of Combinatorics* 18(1), P14 (2011). DOI: 10.37236/501.

[9] The FLINT development team, "Hermite and Smith Normal Forms," software and algorithm documentation. `https://fredrikj.net/fdoc/html/algorithms/linear-algebra/hermite-smith.html`.

[10] W. Steeman, *BPI Challenge 2013, Incidents*, 4TU.ResearchData. DOI: 10.4121/500573e6-accc-4b0c-9576-aa5468b10cee.

[11] I. Verenich, *Helpdesk*, Mendeley Data, version 1 (2016). DOI: 10.17632/39bp3vv62t.1.